CSE373, MAT373: Analysis of Algorithms, Spring 2021 **Date:** May 11, 2021

Final Exam (Solutions)

(5:30 PM – 8:00 PM : 150 Minutes)

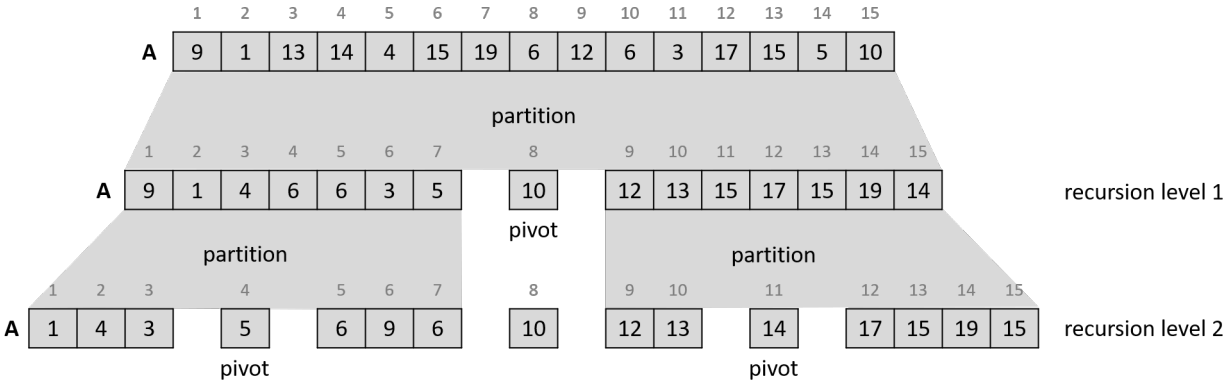
- This exam will account for 45% of your overall grade.
- There are six (6) questions, worth 250 points in total. Please answer all of them.
- This is a *closed book*, *closed notes* exam. *No cheat sheets* are allowed.
- You are allowed to *use scratch papers* for your calculations.
- You are *not allowed to use your own calculator*. A scientific calculator will be available inside the Respondus Lockdown Browser.

Good Luck!

Question	Parts	Points
1. Partitioning	(a)–(b)	$2 \times 8 + 4 = 20$
2. Heaps	(a)–(h)	$5 \times 8 = 40$
3. Dijkstra's SSSP	(a)–(l)	$5 + 5 \times 10 + 10 = 65$
4. Kruskal's MST	(a)–(j)	$5 \times 10 = 50$
5. Matrix Chain Multiplication	(a)–(c)	$20 + 10 + 15 = 45$
6. Complexities	(a)–(d)	$7.5 \times 4 = 30$
Total		250

Question 1. [20 Points] Partitioning. Recall the in-place partitioning algorithm we learnt in the class (before midterm) which is used by quicksort to partition the numbers in the array or subarray it is working on around the pivot. We always chose the last element of the given array/subarray as the pivot.

Given an array $A[1:15]$ as an input the following example shows the state of A after partitioning in the first two levels of recursion of quicksort.



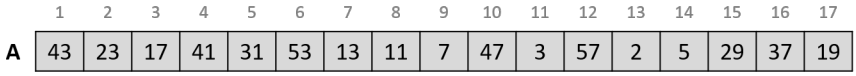
You can describe the state of A after partitioning in recursion level 1 as:

$$[\ 9,\ 1,\ 4,\ 6,\ 6,\ 3,\ 5\ ]\ \ 10\ \ [\ 12,\ 13,\ 15,\ 17,\ 15,\ 19,\ 14\ ]$$

Then after partitioning in recursion level 2 as:

$$[\ [\ 1,\ 4,\ 3\ ]\ \ 5\ \ [\ 6,\ 9,\ 6\ ]\ ]\ \ 10\ \ [\ [\ 12,\ 13\ ]\ \ 14\ \ [\ 17,\ 15,\ 19,\ 15\ ]\ ]$$

1(a) **[16 Points]** Given the following array $A[1:17]$ of numbers describe its state after partitioning in recursion levels 1 and 2.



1(b) **[4 Points]** What is the worst-case time complexity of the partitioning algorithm in terms of the input size n? Simply state the bound. No derivation/proof is necessary. If your bound includes Θ, simply use text "`Theta`" instead of the math symbol Θ.

Solution.

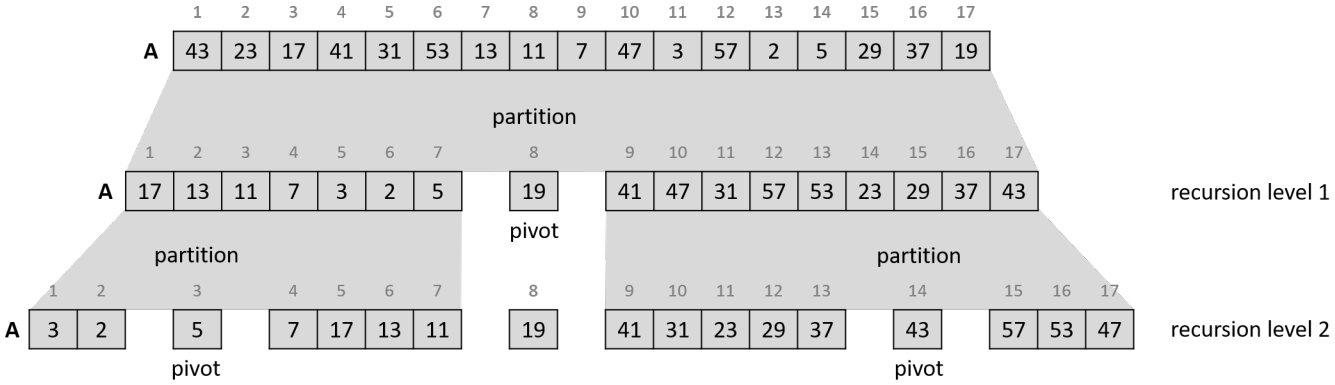
1(a) [**16 Points**] Given the following array $A[1:17]$ of numbers describe its state after partitioning in recursion levels 1 and 2.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
A	43	23	17	41	31	53	13	11	7	47	3	57	2	5	29	37	19

Recursion level 1: [17, 13, 11, 7, 3, 2, 5] 19 [41, 47, 31, 57, 53, 23, 29, 37, 43]

Recursion level 2: [[3, 2] 5 [7, 17, 13, 11]] 19 [[41, 31, 23, 29, 37] 43 [57, 53, 47]]

The solution above corresponds to the following diagram. You do not need to include this diagram in your answer.



1(b) [**4 Points**] What is the worst-case time complexity of the partitioning algorithm in terms of the input size n? Simply state the bound. No derivation/proof is necessary. If your bound includes Θ, simply use text "`Theta`" instead of the math symbol Θ.

The worst-case time complexity is `Theta`(n).

Question 2. [40 Points] Heaps. A max-heap H_{max} supports the following major operations.

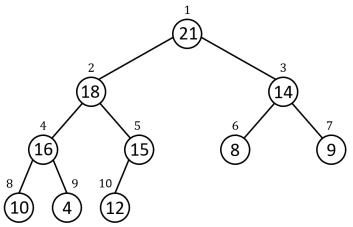
INSERT(H_{max}, k)**:** Insert an item with key k into H_{max}.

INCREASE-KEY(H_{max}, i, k)**:** Increase the key of the item at location i to k assuming $k \geq$ current key of the item.

EXTRACT-MAX(H_{max})**:** Delete an item with the largest key from H_{max} and return it.

A min-heap H_{min}, on the other hand, supports INSERT(H_{min}, k), DECREASE-KEY(H_{min}, i, k), and EXTRACT-MIN(H_{min}).

Now consider the following max-heap H_{max} on 10 items. The number inside each node is the key of an item and the number outside is the current index of that item in the heap.



The state of the heap above in array format can be written down as follows, where for $1 \leq i \leq 10$, the key at index i of the heap is written as the i-th entry of the array:

$$21,\ 18,\ 14,\ 16,\ 15,\ 8,\ 9,\ 10,\ 4,\ 12$$

Now suppose the following array $A[1:14]$ of fourteen (14) numbers is given to you.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
$A[1:14]$:	27	38	20	25	49	15	36	31	30	40	32	50	35	42

After each of the following operations write down the resulting heap in the array format. No need to include any diagrams in your answer.

2(a) [**5 Points**] Build a max-heap H_{max} from $A[1:14]$.

2(b) [**5 Points**] Perform EXTRACT-MAX(H_{max}).

2(c) [**5 Points**] Perform INCREASE-KEY(H_{max}, 12, 51).

2(d) [**5 Points**] Perform INSERT(H_{max}, 55).

2(e) [**5 Points**] Perform EXTRACT-MAX(H_{max}).

2(f) [**5 Points**] Build a min-heap H_{min} from $A[1:14]$.

2(g) [**5 Points**] Perform EXTRACT-MIN(H_{min}).

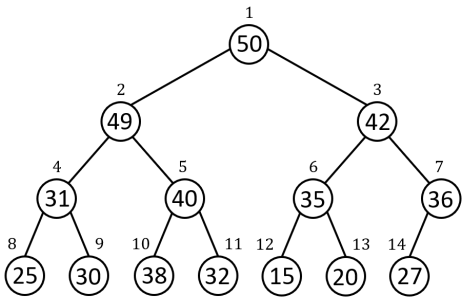
2(h) [**5 Points**] Perform INSERT(H_{min}, 10).

Solution.

2(a) [**5 Points**] Build a max-heap H_{max} from $A[1:14]$.

50, 49, 42, 31, 40, 35, 36, 25, 30, 38, 32, 15, 20, 27

Diagram (no need to include in your answer):

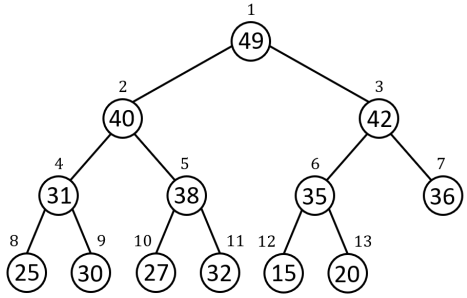


	1	2	3	4	5	6	7	8	9	10	11	12	13	14
$A[1:14]$:	50	49	42	31	40	35	36	25	30	38	32	15	20	27

2(b) [**5 Points**] Perform Extract-Max(H_{max}).

49, 40, 42, 31, 38, 35, 36, 25, 30, 27, 32, 15, 20

Diagram (no need to include in your answer):

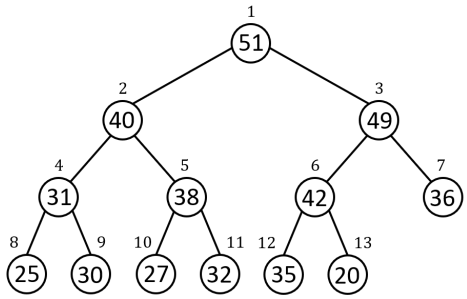


	1	2	3	4	5	6	7	8	9	10	11	12	13
$A[1:13]$:	49	40	42	31	38	35	36	25	30	27	32	15	20

2(*c*) [**5 Points**] Perform INCREASE-KEY(H_{max}, 12, 51).

51, 40, 49, 31, 38, 42, 36, 25, 30, 27, 32, 35, 20

Diagram (no need to include in your answer):

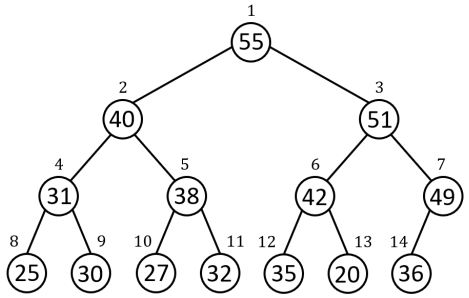


	1	2	3	4	5	6	7	8	9	10	11	12	13
$A[1:13]$:	51	40	49	31	38	42	36	25	30	27	32	35	20

2(*d*) [**5 Points**] Perform INSERT(H_{max}, 55).

55, 40, 51, 31, 38, 42, 49, 25, 30, 27, 32, 35, 20, 36

Diagram (no need to include in your answer):

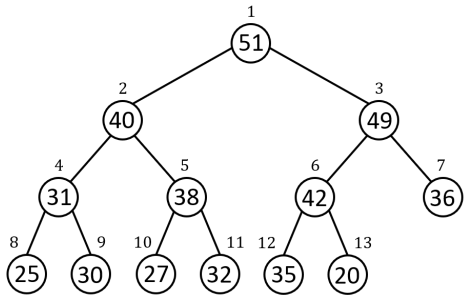


	1	2	3	4	5	6	7	8	9	10	11	12	13	14
$A[1:14]$:	55	40	51	31	38	42	49	25	30	27	32	35	20	36

2(e) [**5 Points**] Perform EXTRACT-MAX(H_{max}).

51, 40, 49, 31, 38, 42, 36, 25, 30, 27, 32, 35, 20

Diagram (no need to include in your answer):

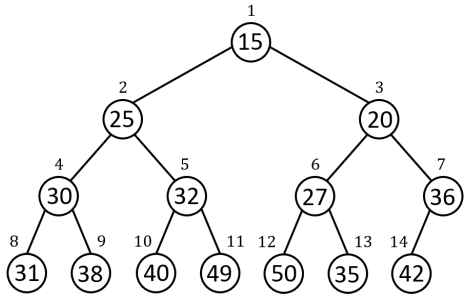


	1	2	3	4	5	6	7	8	9	10	11	12	13
$A[1:13]$:	51	40	49	31	38	42	36	25	30	27	32	35	20

2(f) [**5 Points**] Build a min-heap H_{min} from $A[1:14]$.

15, 25, 20, 30, 32, 27, 36, 31, 38, 40, 49, 50, 35, 42

Diagram (no need to include in your answer):

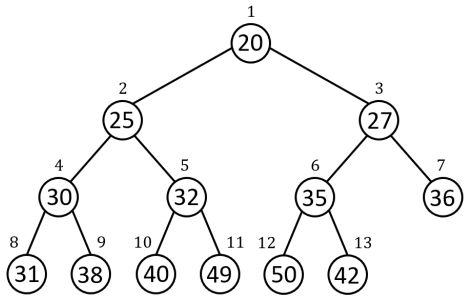


	1	2	3	4	5	6	7	8	9	10	11	12	13	14
$A[1:14]$:	15	25	20	30	32	27	36	31	38	40	49	50	35	42

2(g) [**5 Points**] Perform EXTRACT-MIN(H_{min}).

20, 25, 27, 30, 32, 35, 36, 31, 38, 40, 49, 50, 42

Diagram (no need to include in your answer):

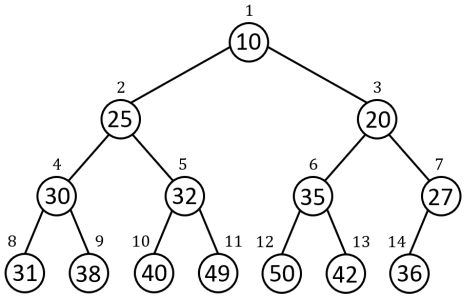


	1	2	3	4	5	6	7	8	9	10	11	12	13
$A[1:13]$:	20	25	27	30	32	35	36	31	38	40	49	50	42

2(h) [**5 Points**] Perform INSERT(H_{min}, 10).

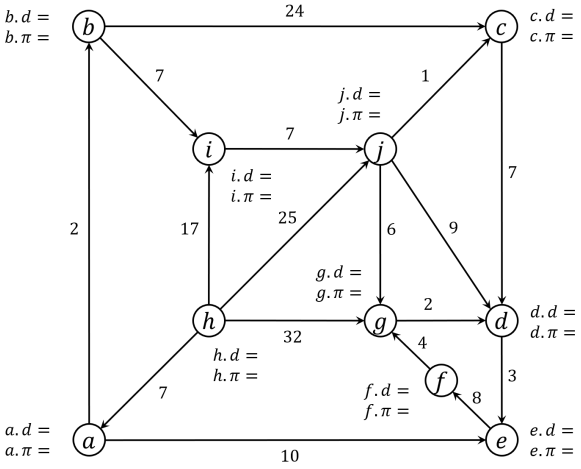
10, 25, 20, 30, 32, 35, 27, 31, 38, 40, 49, 50, 42, 36

Diagram (no need to include in your answer):



	1	2	3	4	5	6	7	8	9	10	11	12	13	14
$A[1:14]$:	10	25	20	30	32	35	27	31	38	40	49	50	42	36

Question 3. [**65 Points**] **Dijkstra's SSSP.** Recall Dijkstra's Single-Source Shortest Paths (SSSP) algorithm we saw in the class. This task asks you for a step-by-step demonstration of how the algorithm finds shortest paths from a given source vertex to all other vertices of the following weighted directed graph.



We will treat vertex h as the given source vertex. Recall that the algorithm first initializes the d and π fields of each node of the graph, and then in each step finalizes the d value of a suitable vertex and relaxes the outgoing edges of the vertex.

In your answers you can write "`pi`" instead of π.

3(a) [**5 Points**] Initialization: What are the initial values of the d and π fields of each vertex?

3(b) [**5 Points**] Step 1: Write down the 1st vertex to be finalized and the vertices whose d and π fields change (along with their new d and π values) as a result of relaxation.

3(c) [**5 Points**] Step 2: Repeat 3(b) for the 2nd vertex to be finalized.

3(d) [**5 Points**] Step 3: Repeat 3(b) for the 3rd vertex to be finalized.

3(e) [**5 Points**] Step 4: Repeat 3(b) for the 4th vertex to be finalized.

3(f) [**5 Points**] Step 5: Repeat 3(b) for the 5th vertex to be finalized.

3(g) [**5 Points**] Step 6: Repeat 3(b) for the 6th vertex to be finalized.

3(h) [**5 Points**] Step 7: Repeat 3(b) for the 7th vertex to be finalized.

3(i) [**5 Points**] Step 8: Repeat 3(b) for the 8th vertex to be finalized.

3(j) [**5 Points**] Step 9: Repeat 3(b) for the 9th vertex to be finalized.

3(k) [**5 Points**] Step 10: Repeat 3(b) for the 10th vertex to be finalized.

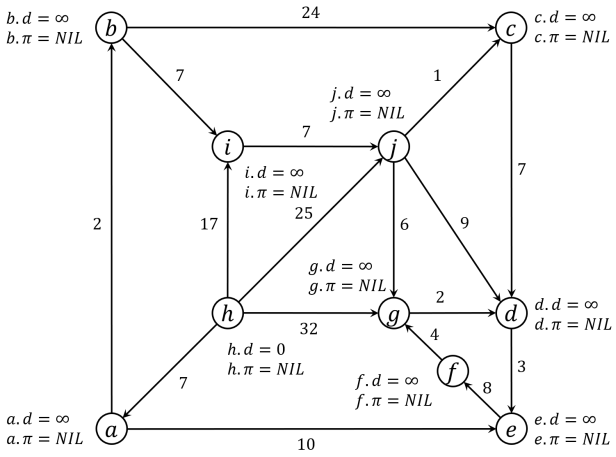
3(l) [**10 Points**] By examining the final d and π fields of the vertices of the graph, write down the shortest distance and the corresponding shortest path from the source vertex (i.e., vertex h) to each of the following vertices: c, d, g, and j.

Solution.

3(a) [**5 Points**] Initialization: What are the initial values of the d and π fields of each vertex?

The $(d, \texttt{pi})$ fields of vertex h are set to $(0, \texttt{NIL})$, and those of all other vertices are set to $(\texttt{infinity}, \texttt{NIL})$.

Diagram (no need to include in your answer):

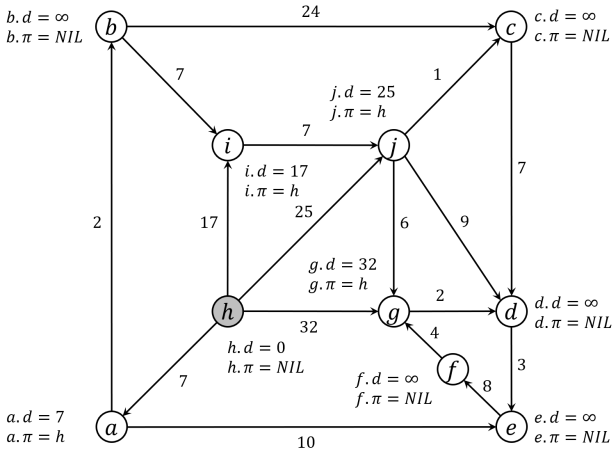


3(b) [**5 Points**] Step 1: Write down the 1st vertex to be finalized and the vertices whose d and π fields change (along with their new d and π values) as a result of relaxation.

Finalized: h

Changed $(d, \texttt{pi})$: a $(7, h)$, g $(32, h)$, i $(17, h)$, j $(25, h)$

Diagram (no need to include in your answer):

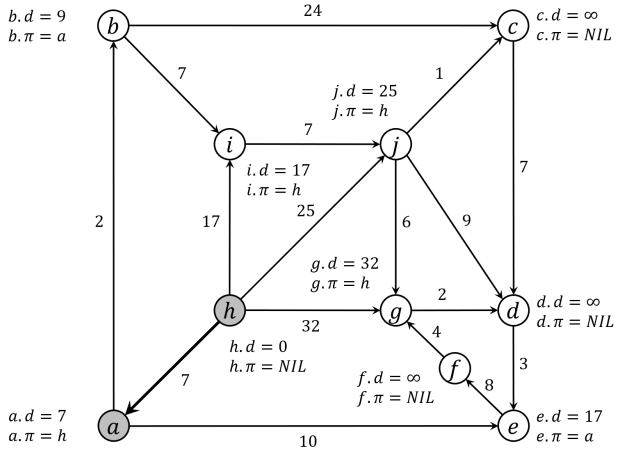


3(c) [**5 Points**] Step 2: Repeat 3(b) for the 2nd vertex to be finalized.

Finalized: a

Changed $(d, \texttt{pi})$: b $(9, a)$, e $(17, a)$

Diagram (no need to include in your answer):

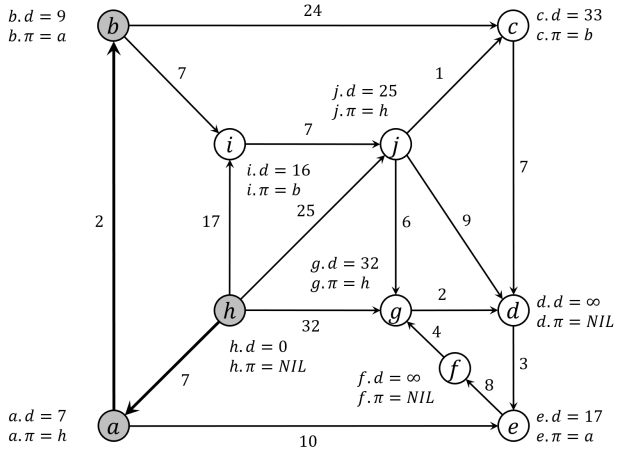


3(d) [**5 Points**] Step 3: Repeat 3(b) for the 3rd vertex to be finalized.

Finalized: b

Changed $(d, \texttt{pi})$: c $(33, b)$, i $(16, b)$

Diagram (no need to include in your answer):

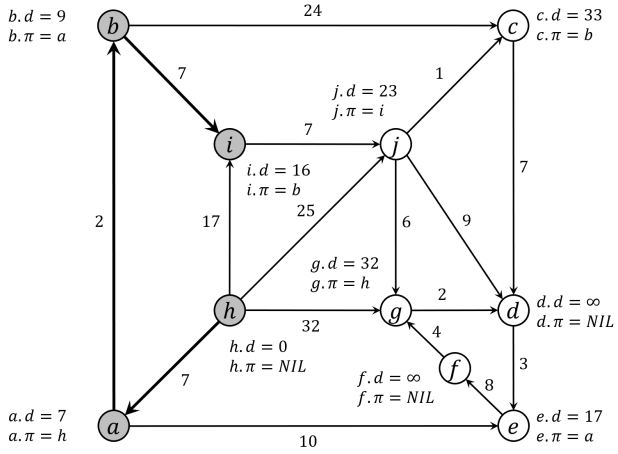


3(e) [**5 Points**] Step 4: Repeat 3(b) for the 4th vertex to be finalized.

Finalized: i

Changed $(d, \texttt{pi})$: j $(23, i)$

Diagram (no need to include in your answer):

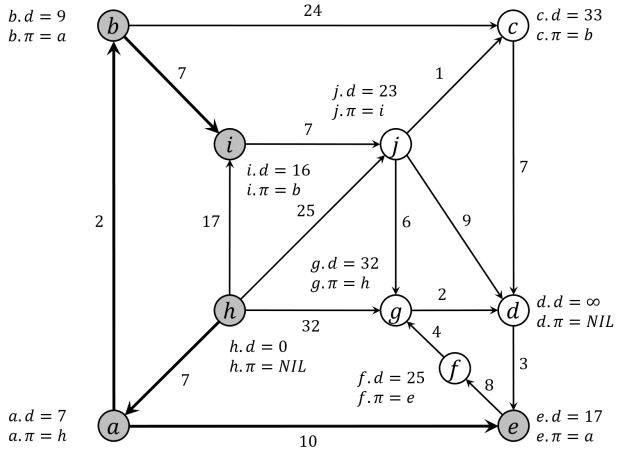


3(f) [**5 Points**] Step 5: Repeat 3(b) for the 5th vertex to be finalized.

Finalized: e

Changed $(d, \texttt{pi})$: f $(25, e)$

Diagram (no need to include in your answer):

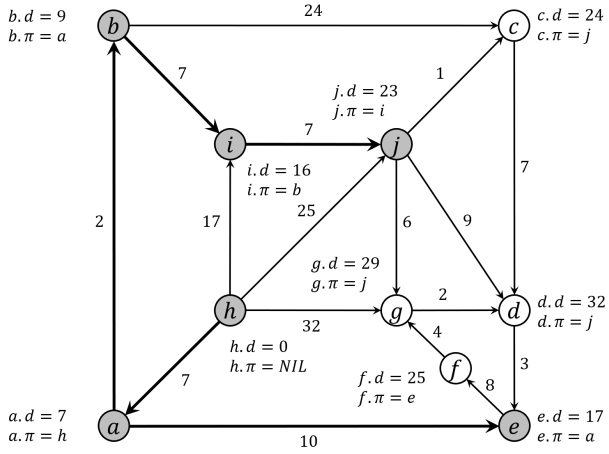


3(g) [**5 Points**] Step 6: Repeat 3(b) for the 6th vertex to be finalized.

Finalized: j

Changed $(d, \texttt{pi})$: c $(24, j)$, d $(32, j)$, g $(29, j)$

Diagram (no need to include in your answer):

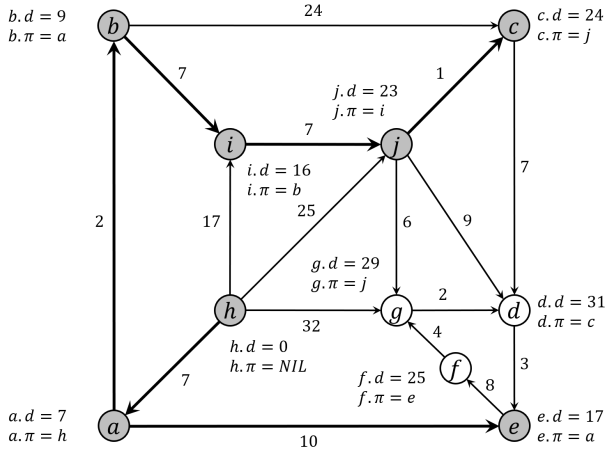


3(h) [**5 Points**] Step 7: Repeat 3(b) for the 7th vertex to be finalized.

Finalized: c

Changed $(d, \texttt{pi})$: d $(31, c)$

Diagram (no need to include in your answer):

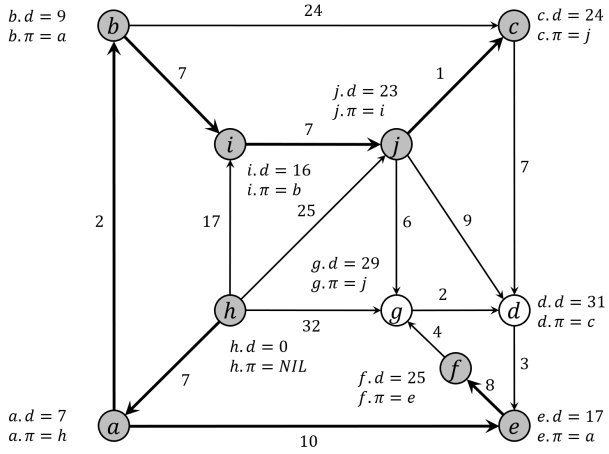


3(i) [**5 Points**] Step 8: Repeat 3(b) for the 8th vertex to be finalized.

Finalized: f

Changed (d, `pi`): none

Diagram (no need to include in your answer):

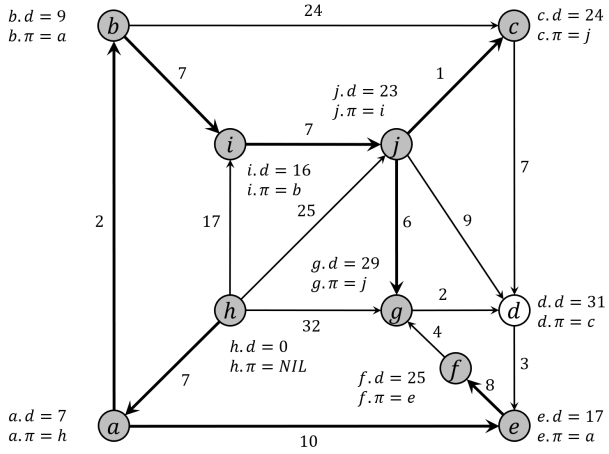


3(j) [**5 Points**] Step 9: Repeat 3(b) for the 9th vertex to be finalized.

Finalized: g

Changed (d, `pi`): none

Diagram (no need to include in your answer):

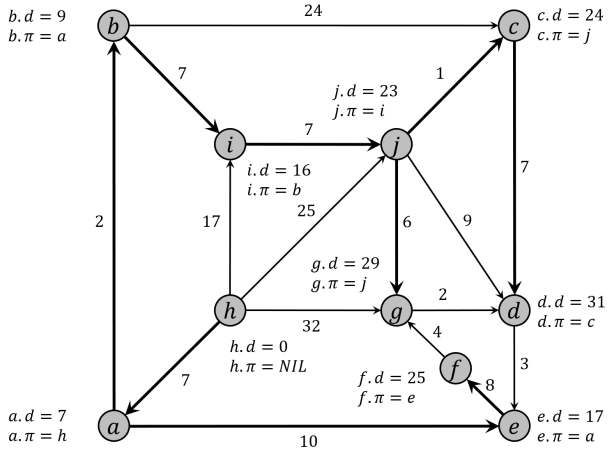


3(k) [**5 Points**] STEP 10: Repeat 3(b) for the 10th vertex to be finalized.

Finalized: d

Changed (d, `pi`): none

Diagram (no need to include in your answer):



3(l) [**10 Points**] By examining the final d and π fields of the vertices of the graph, write down the shortest distance and the corresponding shortest path from the source vertex (i.e., vertex h) to each of the following vertices: c, d, g, and j.

Vertex c: distance 24, path $h -> a -> b -> i -> j -> c$

Vertex d: distance 31, path $h -> a -> b -> i -> j -> c -> d$

Vertex g: distance 29, path $h -> a -> b -> i -> j -> g$

Vertex j: distance 23, path $h -> a -> b -> i -> j$

NOTE: Another possible answer to 3(j) is:

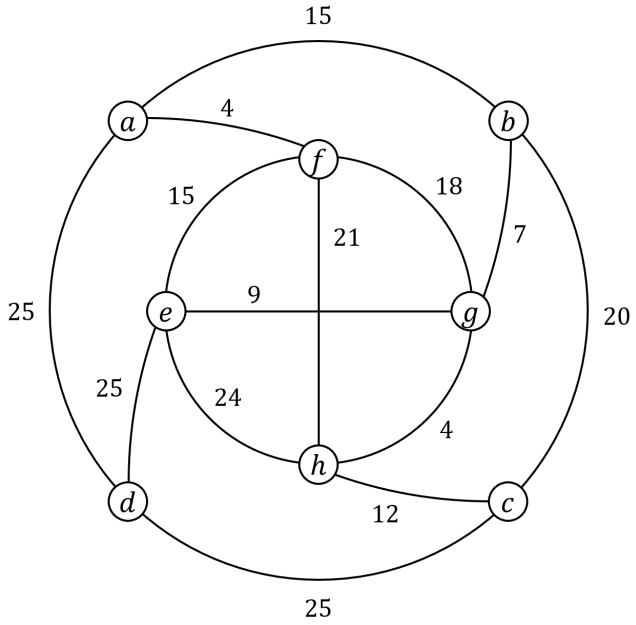
Finalized: g

Changed (d, `pi`): d $(31, g)$

In that case part of answer to 3(l) will change as follows:

Vertex d: distance 31, path $h -> a -> b -> i -> j -> g -> d$

Question 4. [**50 Points**] **Kruskal's MST.** Recall Kruskal's Minimum Spanning Tree (MST) algorithm we saw in the class. This task asks you for a step-by-step demonstration of how the algorithm finds an MST of the following weighted undirected graph.



Recall that the algorithm examines the edges one by one in a predetermined sequence. In each step the algorithm examines one edge, and decides to either include it in the MST or discard it.

4(a) [**5 Points**] Give an order in which the algorithm may examine the edges of the graph.

4(b) [**5 Points**] In Kruskal's algorithm when do you decide not to include an edge to the MST?

4(c) [**5 Points**] Write down the 1st edge to be included in the MST.

4(d) [**5 Points**] Write down the 2nd edge to be included in the MST.

4(e) [**5 Points**] Write down the 3rd edge to be included in the MST.

4(f) [**5 Points**] Write down the 4th edge to be included in the MST.

4(g) [**5 Points**] Write down the 5th edge to be included in the MST.

4(h) [**5 Points**] Write down the 6th edge to be included in the MST.

4(i) [**5 Points**] Write down the 7th edge to be included in the MST.

4(j) [**5 Points**] What is the weight of the MST you have just computed?

Solution.

4(a) [**5 Points**] Give an order in which the algorithm may examine the edges of the graph.

The edges will be considered in the following order (in nondecreasing order of weight):

$(a, f),\ (g, h),\ (b, g),\ (e, g),\ (c, h),\ (a, b),\ (e, f),\ (f, g),\ (b, c),\ (f, h),\ (e, h),\ (a, d),\ (c, d),\ (d, e)$

Any of the following orders are acceptable, where the edges inside curly braces can be considered in any order.

$\{(a, f),\ (g, h)\},\ (b, g),\ (e, g),\ (c, h),\ \{(a, b),\ (e, f)\},\ (f, g),\ (b, c),\ (f, h),\ (e, h),$
$\{(a, d),\ (c, d),\ (d, e)\}$

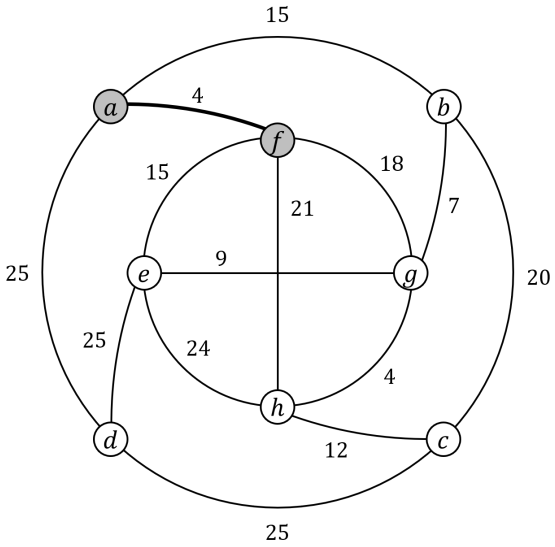
4(b) [**5 Points**] In Kruskal's algorithm when do you decide not to include an edge to the MST?

We decide not to add an edge to the MST provided it forms a cycle with all or some of the edges already included in the MST.

4(c) [**5 Points**] Write down the 1st edge to be included in the MST.

1st edge to be included: (a, f)

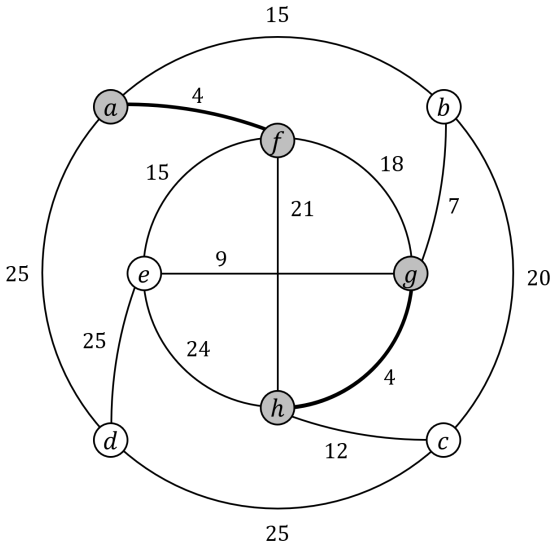
Diagram (no need to include in your answer):



4(d) [**5 Points**] Write down the 2nd edge to be included in the MST.

1st edge to be included: (g, h)

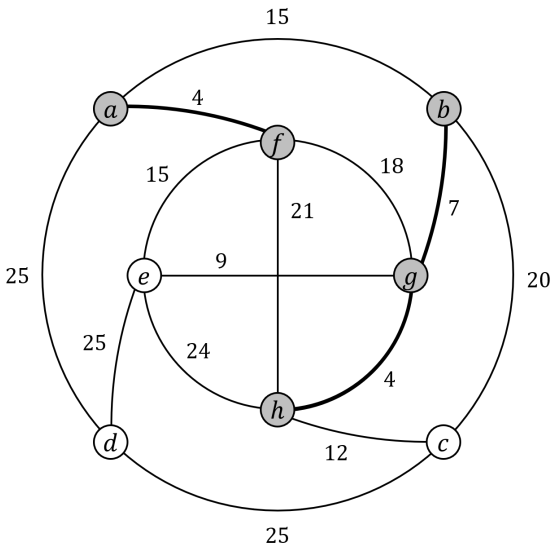
Diagram (no need to include in your answer):



4(e) [**5 Points**] Write down the 3rd edge to be included in the MST.

1st edge to be included: (b, g)

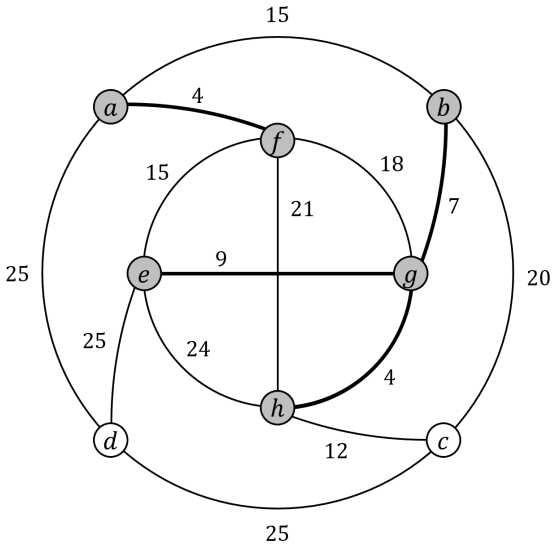
Diagram (no need to include in your answer):



4(f) [**5 Points**] Write down the 4th edge to be included in the MST.

1st edge to be included: (e, g)

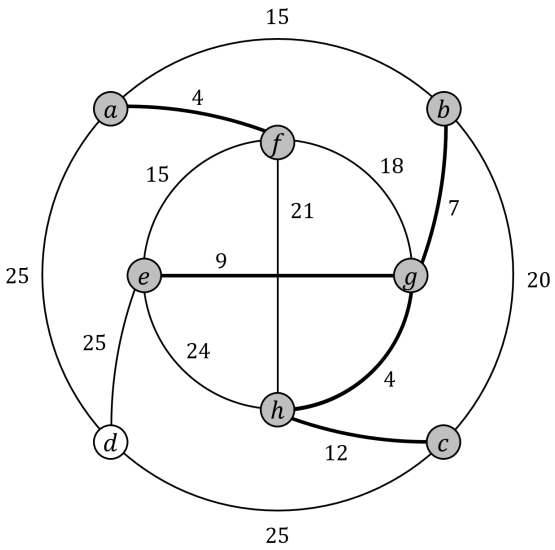
Diagram (no need to include in your answer):



4(g) [**5 Points**] Write down the 5th edge to be included in the MST.

1st edge to be included: (c, h)

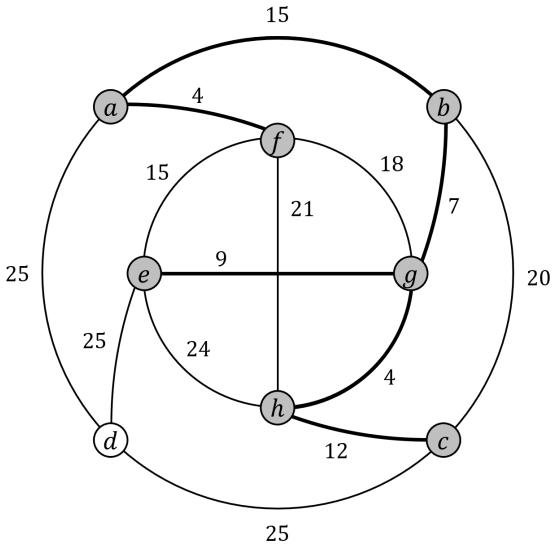
Diagram (no need to include in your answer):



4(h) [**5 Points**] Write down the 6th edge to be included in the MST.

1st edge to be included: (a, b)

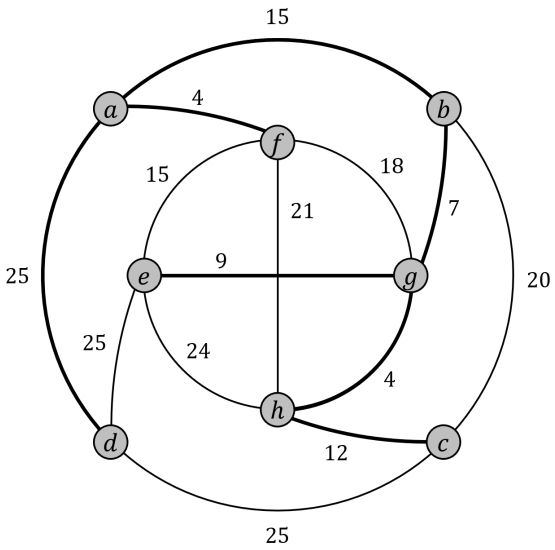
Diagram (no need to include in your answer):



4(i) [**5 Points**] Write down the 7th edge to be included in the MST.

1st edge to be included: (a, d)

Diagram (no need to include in your answer):



4(j) [**5 Points**] What is the weight of the MST you have just computed?

Total weight of the MST: $w(a,f)+w(g,h)+w(b,g)+w(e,g)+w(c,h)+w(a,b)+w(a,d) = 4+4+7+9+12+15+25 = 76$.

NOTE: All twelve (12) different sequences in which one can add the edges the MST.

$4(c):\ (a,f);\ \ 4(d):\ (g,h),\ \ 4(e):\ (b,g),\ \ 4(f):\ (e,g),\ \ 4(g):\ (c,h),\ \ 4(h):\ (a,b),\ \ 4(i):\ (a,d)$

$4(c):\ (a,f);\ \ 4(d):\ (g,h),\ \ 4(e):\ (b,g),\ \ 4(f):\ (e,g),\ \ 4(g):\ (c,h),\ \ 4(h):\ (a,b),\ \ 4(i):\ (c,d)$

$4(c):\ (a,f);\ \ 4(d):\ (g,h),\ \ 4(e):\ (b,g),\ \ 4(f):\ (e,g),\ \ 4(g):\ (c,h),\ \ 4(h):\ (a,b),\ \ 4(i):\ (e,d)$

$4(c):\ (a,f);\ \ 4(d):\ (g,h),\ \ 4(e):\ (b,g),\ \ 4(f):\ (e,g),\ \ 4(g):\ (c,h),\ \ 4(h):\ (e,f),\ \ 4(i):\ (a,d)$

$4(c):\ (a,f);\ \ 4(d):\ (g,h),\ \ 4(e):\ (b,g),\ \ 4(f):\ (e,g),\ \ 4(g):\ (c,h),\ \ 4(h):\ (e,f),\ \ 4(i):\ (c,d)$

$4(c):\ (a,f);\ \ 4(d):\ (g,h),\ \ 4(e):\ (b,g),\ \ 4(f):\ (e,g),\ \ 4(g):\ (c,h),\ \ 4(h):\ (e,f),\ \ 4(i):\ (e,d)$

$4(c):\ (g,h);\ \ 4(d):\ (a,f),\ \ 4(e):\ (b,g),\ \ 4(f):\ (e,g),\ \ 4(g):\ (c,h),\ \ 4(h):\ (a,b),\ \ 4(i):\ (a,d)$

$4(c):\ (g,h);\ \ 4(d):\ (a,f),\ \ 4(e):\ (b,g),\ \ 4(f):\ (e,g),\ \ 4(g):\ (c,h),\ \ 4(h):\ (a,b),\ \ 4(i):\ (c,d)$

$4(c):\ (g,h);\ \ 4(d):\ (a,f),\ \ 4(e):\ (b,g),\ \ 4(f):\ (e,g),\ \ 4(g):\ (c,h),\ \ 4(h):\ (a,b),\ \ 4(i):\ (e,d)$

$4(c):\ (g,h);\ \ 4(d):\ (a,f),\ \ 4(e):\ (b,g),\ \ 4(f):\ (e,g),\ \ 4(g):\ (c,h),\ \ 4(h):\ (e,f),\ \ 4(i):\ (a,d)$

$4(c):\ (g,h);\ \ 4(d):\ (a,f),\ \ 4(e):\ (b,g),\ \ 4(f):\ (e,g),\ \ 4(g):\ (c,h),\ \ 4(h):\ (e,f),\ \ 4(i):\ (c,d)$

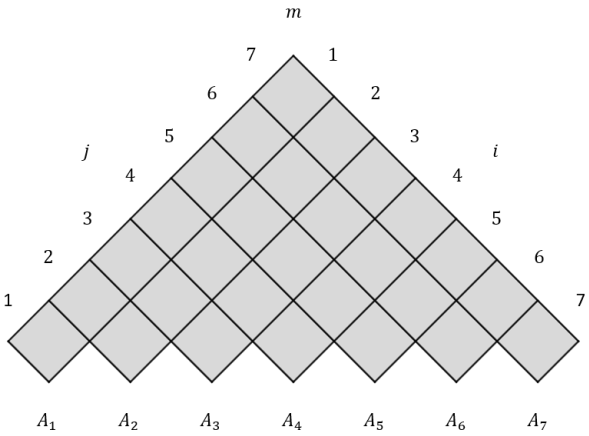
$4(c):\ (g,h);\ \ 4(d):\ (a,f),\ \ 4(e):\ (b,g),\ \ 4(f):\ (e,g),\ \ 4(g):\ (c,h),\ \ 4(h):\ (e,f),\ \ 4(i):\ (e,d)$

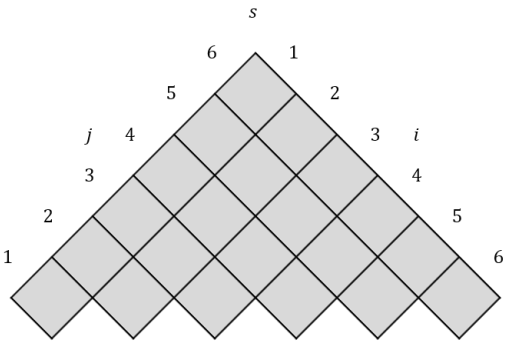
Question 5. [**45 Points**] **Matrix Chain Multiplication.** Recall that the matrix-chain multiplication problem is as follows: given a chain $\langle A_1, A_2, \ldots, A_n \rangle$ of n matrices, where for $i = 1, 2, \ldots, n$ matrix A_i has dimension $p_{i-1} \times p_i$, fully parenthesize the product $A_1 \times A_2 \times \ldots \times A_n$ in a way that minimizes the number of scalar multiplications.

In this task, you are given the following chain of seven (7) matrices.

matrix	A_1	A_2	A_3	A_4	A_5	A_6	A_7
dimension	1×8	8×5	5×2	2×6	6×3	3×4	4×1

Your task is to fill out the following two tables the way we saw in the class.





5(a) [**20 Points**] Fill out the tables given above.

For the m table: for each $k \in [1, 7]$, write down all entries $m[i, j]$ with $j - i = 7 - k$ from on line k. Put the entries on the same line in increasing order of i values.

For the s table: for each $k \in [1, 6]$, write down all entries $s[i, j]$ with $j - i = 7 - k$ from on line k. Put the entries on the same line in increasing order of i values.

5(b) [**10 Points**] What is the smallest number of scalar multiplications needed to multiply the seven input matrices? Which entry of which table stores this value?

5(c) [**15 Points**] Write down the parenthesization of $A_1 \times A_2 \times \ldots \times A_7$ that minimizes the number of scalar multiplications needed to multiply them. Which table do you extract it from? What is the size of the final product matrix?

Solution.

5(a) [**20 Points**] Fill out the tables given above.

For the m table: for each $k \in [1, 7]$, write down all entries $m[i, j]$ with $j - i = 7 - k$ from on line k. Put the entries on the same line in increasing order of i values.

For the s table: for each $k \in [1, 6]$, write down all entries $s[i, j]$ with $j - i = 7 - k$ from on line k. Put the entries on the same line in increasing order of i values.

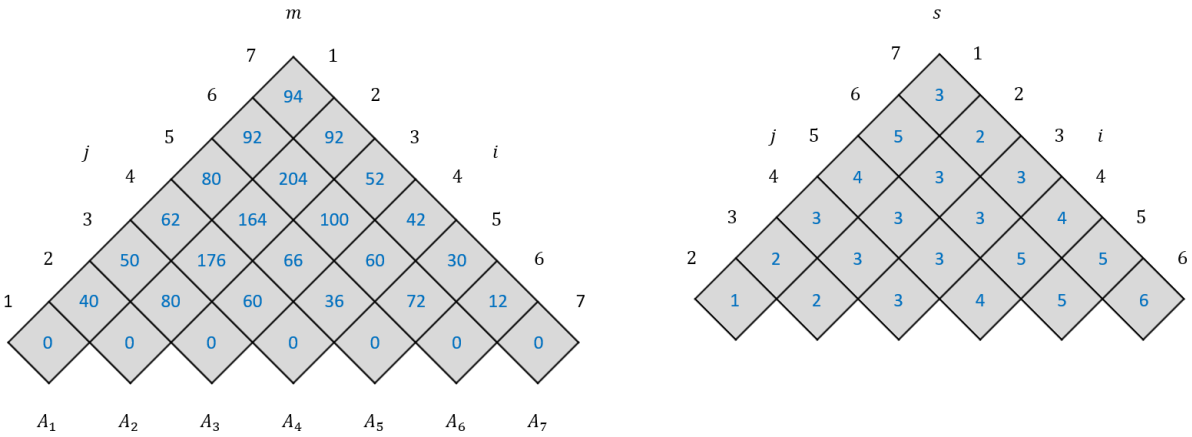
Table m:

94						
92	92					
80	204	52				
62	164	100	42			
50	176	66	60	30		
40	80	60	36	72	12	
0	0	0	0	0	0	0

Table s:

3					
5	2				
4	3	3			
3	3	3	4		
2	3	3	5	5	
1	2	3	4	5	6

Diagram (no need to include in your answer):



5(b) [**10 Points**] What is the smallest number of scalar multiplications needed to multiply the seven input matrices? Which entry of which table stores this value?

The smallest number of scalar multiplications need: 94.

This number is stored in $m[1, 7]$.

5(c) [**10 Points**] Write down the parenthesization of $A_1 \times A_2 \times \ldots \times A_7$ that minimizes the number of scalar multiplications needed to multiply them. Which table do you extract it from? What is the size of the final product matrix?

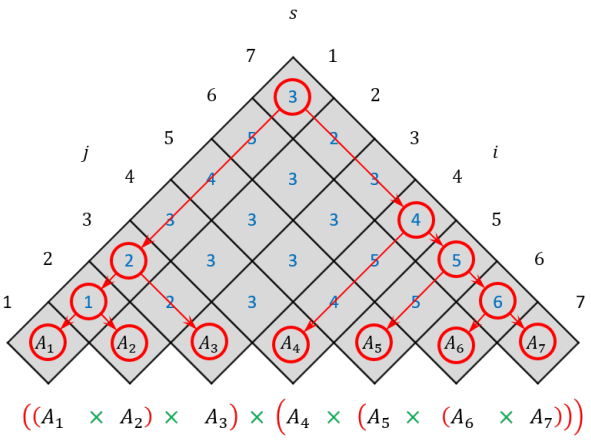
Here is the optimal parenthesization:

`( ( A1 x A2 ) x A3 ) x ( A4 x ( A5 x ( A6 x A7 ) ) ).`

The ordering above is obtained from the s table.

The size of the final product matrix is: 1×1.

Diagram (no need to include in your answer):



Question 6. [30 Points] Complexities. For each of the problems given below, write down which problem we saw in the class it corresponds to. If there is a polynomial-time (or pseudo-polynomial time) algorithm for solving the problem, state the running time of that algorithm. Otherwise write "`Hard`" instead of stating the running time.

No justifications necessary, but you must state what each variable in your stated complexity means (e.g., meanings of m and n in $\mathcal{O}((m+n)\log\log m)$).

6(a) [**7.5 Points**] Suppose you know the friends list of every person on Facebook. Assuming that friendship is commutative (i.e., a is on b's friends list implies that b is on a's friends list and vice versa), you want to find out if there exists a group of k people on Facebook such that every person in that group is on the friends list of every other person in the group.

6(b) [**7.5 Points**] Suppose you are running a car sharing business and for each car in your fleet you get a list of reservation requests every morning. Each request has a starting time and an ending time. You want to accommodate as many reservation requests as possible for every car under the obvious constraint that no two of them can overlap (i.e., starting time of one request must not be earlier than the ending time of the request just before it).

6(c) [**7.5 Points**] A delivery truck driver has a list of addresses to which packages need to be delivered on a given day. The driver starts from the local delivery facility in the morning and must come back to the facility after delivering all packages. He/she knows the driving time between the delivery facility and every address on the list as well as between every pair of addresses on the list. Now assuming that the driver can work for at most t hours on a day, he/she wants to figure out if it is possible to visit every address on the list exactly once to deliver the packages and come back to the facility within that time limit.

6(d) [**7.5 Points**] There are n questions in a test. The i-th question takes t_i minutes to answer and is worth p_i points (say, you score all p_i points if you answer the question). Suppose you have only T minutes to answer the questions. You want to figure out which subset of the questions you should answer to score the maximum number of points within the given time limit.

SOLUTION.

6(a) [**7.5 Points**] Suppose you know the friends list of every person on Facebook. Assuming that friendship is commutative (i.e., a is on b's friends list implies that b is on a's friends list and vice versa), you want to find out if there exists a group of k people on Facebook such that every person in that group is on the friends list of every other person in the group.

Corresponds to the *Clique Problem* which is `Hard`.

6(b) [**7.5 Points**] Suppose you are running a car sharing business and for each car in your fleet you get a list of reservation requests every morning. Each request has a starting time and an ending time. You want to accommodate as many reservation requests as possible for every car under the obvious constraint that no two of them can overlap (i.e., starting time of one request must not be earlier than the ending time of the request just before it).

Corresponds to the *Activity Selection Problem* which can be solved in $\mathcal{O}(n \log n)$ time, where n is the number of activities (i.e., reservation requests).

[Note: If the activities are already sorted in non-decreasing order of finish times then it can be solved in $\mathcal{O}(n)$ time.]

6(c) [**7.5 Points**] A delivery truck driver has a list of addresses to which packages need to be delivered on a given day. The driver starts from the local delivery facility in the morning and must come back to the facility after delivering all packages. He/she knows the driving time between the delivery facility and every address on the list as well as between every pair of addresses on the list. Now assuming that the driver can work for at most t hours on a day, he/she wants to figure out if it is possible to visit every address on the list exactly once to deliver the packages and come back to the facility within that time limit.

Corresponds to the *Traveling Salesman Problem* which is `Hard`.

6(d) [**7.5 Points**] There are n questions in a test. The i-th question takes t_i minutes to answer and is worth p_i points (say, you score all p_i points if you answer the question). Suppose you have only T minutes to answer the questions. You want to figure out which subset of the questions you should answer to score the maximum number of points within the given time limit.

Corresponds to the *0-1 Knapsack Problem* which can solved in $\mathcal{O}(nW)$ time, where n is the number objects (= number of questions in this case) and W is the capacity of the knapsack (= total time T in this case). Please note that we are assuming that W (= T) and all weights w_i (= t_i in this case) are integers.